



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 01

NOMBRE COMPLETO: Alfaro Domínguez Patricio

N° de Cuenta: 320051665

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 11/09/2025

CALIFICACIÓN: _____

1.- Generar una pirámide rubik (pyraminx) de 9 **pirámides** por cara.

Cada cara de la pyraminx que se vea de un color diferente y que se vean las separaciones entre instancias (las líneas oscuras son las que permiten diferenciar cada pirámide pequeña)

El código utilizado para la pirámide negra que actúa como base fue:

```
//Negro
model = glm::mat4(1.0);
//otras transformaciones para el objeto
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
// The new translation moves the black pyramid to (0,0,0)
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.3f, 1.3f, 1.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la linea de proyeccion solo se manda una vez a menos que en tiempo de ejecucion
//se programe cambio entre proyeccion ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[1]->RenderMesh(); //dibuja cubo y piramide triangular
```

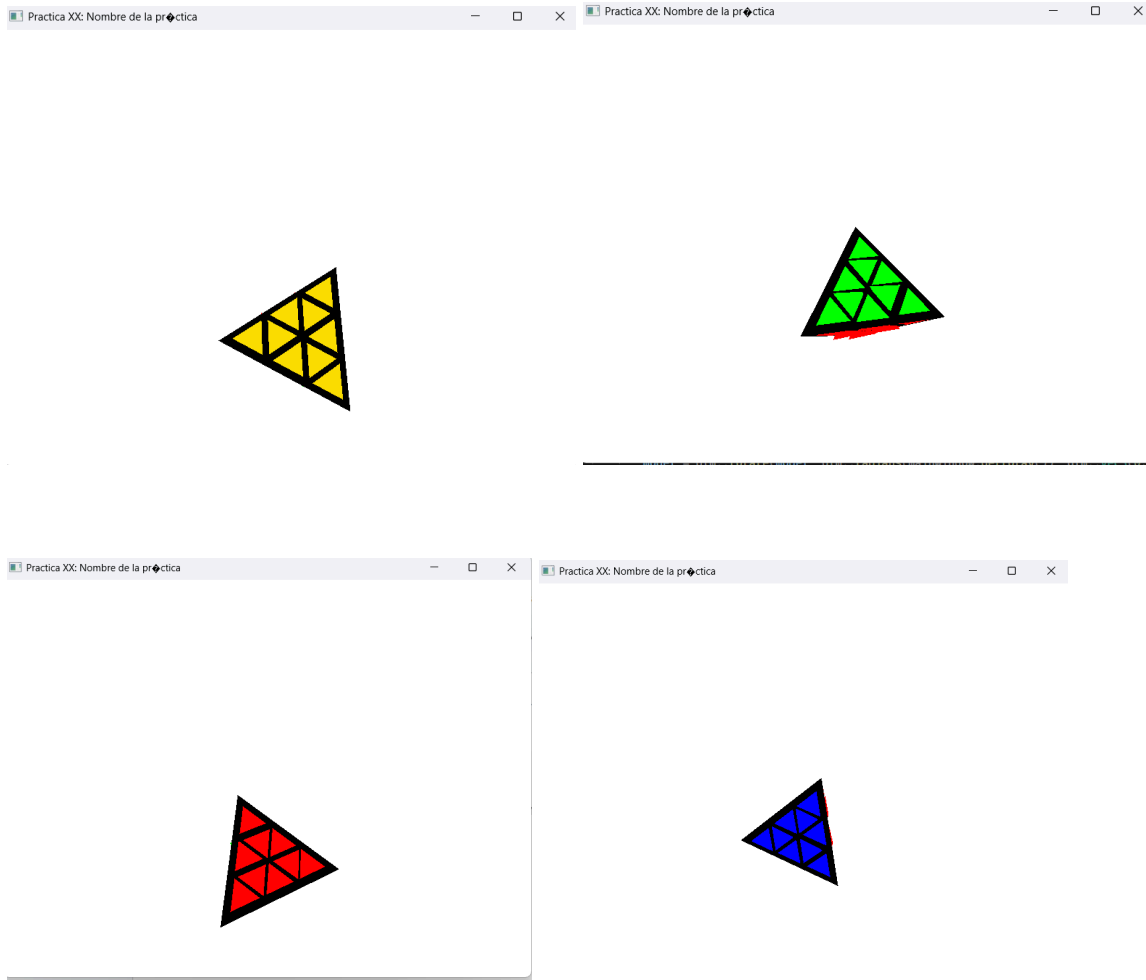
Las pirámides chiquitas recibieron un escalamiento de 0.3 para de esa forma abarcar de forma bonita las caras de la base. Un ejemplo de una de estas figuras es:

```
// amarillo
model = glm::mat4(1.0);
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
// (0.0f, -0.01f, -4.25f) + (0.0f, -0.2025f, 3.8f) = (0.0f, -0.2125f, -0.45f)
model = glm::translate(model, glm::vec3(0.0f, -0.2125f, -0.45f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.98f, 0.88f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh();
```

El rotate se colocó antes de las demás transformaciones geométricas, ya que gracias a esto rota todo antes de posicionarse, permitiendo que todo rote en conjunto.

Resultado:

Conclusión:



En cuanto a dificultades la verdad es que no se me ocurrió como hacerle para utilizar una pirámide nada más para los picos y las aristas, para que los lados compartieran pirámide y solo cambiara el color de las caras. Por eso tuve que utilizar 36 pirámides distintas para obtener el resultado deseado.

Conclusiones:

Esta práctica estuvo interesante ya que aparte de poder jugar con las rotaciones y traslaciones para obtener la figura deseada, también jugó con mi paciencia para colocarlos en las coordenadas que correspondían y que se viera bonito.

Fuentes:

LearnOpenGL - Transformations. (s. f.).

<https://learnopengl.com/Getting-started/Transformations>